# Data Structures and Algorithms — Lab 5

Stack, Queue, Priority Queue, and Hash Table
Order-Based and Hash-Based Data Structures

School of Computer Science and Engineering

## Lab Theme

In previous labs, you practiced solving array and string problems using algorithmic patterns such as Two Pointers, Sliding Window, Prefix Sum, and Hashing. Those techniques help reduce repeated work by moving boundaries intelligently or by storing useful past information.

In this lab, you will study several fundamental data structures that organize information in different ways:

- Stack,
- Queue,
- Deque,
- Priority Queue,
- Hash Table,
- HashSet and HashMap.

The main goal is not only to use these data structures, but also to explain why each structure is appropriate for a given problem.

The key question for this lab is:

> *How should data be stored so that the next operation becomes efficient?*

## 1 Learning Outcomes

After completing this lab, you should be able to:

- explain the differences among Stack, Queue, Deque, Priority Queue, and Hash Table;
- use Stack-like and Queue-like structures in Java;
- implement a basic Circular Queue using an array;
- explain how a Priority Queue selects elements based on priority;
- explain the idea of hashing;

- describe how a hash table maps a key to an index;
- explain collision handling using separate chaining;
- use `HashSet` and `HashMap` for fast lookup and frequency counting;
- compare brute-force and optimized approaches;
- analyze time complexity and space complexity clearly.

# 2 General Requirements

## 2.1 Programming Requirements

- Use Java.
- Organize source code by tasks or exercises.
- Use meaningful variable names and readable program structure.
- Your program must compile and run using standard Java commands.
- You may use Java built-in structures for some exercises, but not for the manual implementation exercises.

Useful Java classes include:

```
import java.util.Stack;
import java.util.ArrayDeque;
import java.util.Queue;
import java.util.PriorityQueue;
import java.util.HashSet;
import java.util.HashMap;
import java.util.LinkedList;
```

However, for the Circular Queue and Simple Hash Table exercises, you must implement the required structure manually.

## 2.2 Problem-Solving and Explanation Requirements

For each required exercise, your report should include:

1. problem restatement,
2. input, output, assumptions, and edge cases,
3. brute-force idea, if applicable,
4. optimized idea,
5. selected data structure,
6. what information is stored,
7. why the data structure is suitable,

8. time complexity and space complexity,
9. at least 3 test cases.

During explanation, do **not** explain the code line by line. Instead, focus on the idea.
Ask yourself:

- What does the stack store?
- What does the queue store?
- What determines the priority?
- What is used as the hash key?
- What happens when two keys map to the same index?
- Why is the optimized solution faster?

# 3 Core Concept 1: Stack

## 3.1 Definition

A Stack is a data structure that follows:

**Last-In, First-Out (LIFO)**

The last element inserted is the first element removed.

Common operations:

```
push(x)      // insert x onto the top of the stack
pop()        // remove and return the top element
peek()       // return the top element without removing it
isEmpty()    // check whether the stack is empty
```

## 3.2 Main Intuition

A stack is useful when the most recent unresolved item should be processed first.

Common examples:

- matching parentheses,
- undo operation,
- browser history,
- expression evaluation,
- removing adjacent duplicates.

| Variant | Main Idea |
|---|---|
| Normal Stack | Stores elements in LIFO order. |
| Min Stack | Supports retrieving the minimum value efficiently. |
| Monotonic Stack | Keeps elements in increasing or decreasing order. |
| Stack using Array | Manual implementation using an array. |
| Stack using Linked List | Manual implementation using nodes. |

Table 1: Common stack variants

### 3.3 Stack Variants

For this one-week lab, only the normal Stack is required. Other variants may appear in further practice.

## 4 Core Concept 2: Queue and Deque

### 4.1 Queue Definition

A Queue is a data structure that follows:

**First-In, First-Out (FIFO)**

The first element inserted is the first element removed.

Common operations:

```
enqueue(x)  // insert x at the rear of the queue
dequeue()   // remove and return the front element
front()     // return the front element without removing it
isEmpty()   // check whether the queue is empty
```

### 4.2 Circular Queue

A Circular Queue uses an array, but the front and rear positions wrap around when they reach the end of the array. This avoids wasting space.

Important variables:

```
front
rear
size
capacity
```

Circular movement:

```
rear = (rear + 1) % capacity;
front = (front + 1) % capacity;
```

### 4.3 Deque

A Deque, or double-ended queue, supports insertion and deletion from both ends.

Common operations:

```
addFirst(x)
addLast(x)
removeFirst()
removeLast()
```

A Deque can behave like both a Stack and a Queue.

## 5 Core Concept 3: Priority Queue

### 5.1 Definition

A Priority Queue removes elements based on priority, not insertion order.

In Java, `PriorityQueue` is usually a min-priority queue by default. This means the smallest element is removed first.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(5);
pq.add(2);
pq.add(8);

System.out.println(pq.poll()); // 2
```

### 5.2 Max Priority Queue

To create a max-priority queue in Java:

```
PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> b - a);
```

### 5.3 When to Use Priority Queue

Use a Priority Queue when:

- you need the smallest or largest item repeatedly,
- you need the top `k` items,
- each item has a priority,
- normal FIFO order is not enough.

Common problems:

- Kth largest element,
- Top K frequent elements,

- task scheduling,
- connect ropes with minimum cost,
- merging sorted data.

# 6 Core Concept 4: Hash Table and Hashing

## 6.1 What Is Hashing?

Hashing is a technique that converts a key into an integer value. That integer value is then used to decide where to store the key-value pair in a table.

Basic idea:

```
key -> hash value -> table index
```

A common formula is:

```
index = hash(key) % tableSize;
```

## 6.2 Simple Integer Hashing

For integer keys:

```
int index = key % tableSize;
```

If the key may be negative:

```
int index = Math.abs(key) % tableSize;
```

## 6.3 Simple String Hashing Algorithm

A simple string hashing method is:

```
int hash = 0;

for (int i = 0; i < key.length(); i++) {
    hash = hash * 31 + key.charAt(i);
}
```

Then compress it into a table index:

```
int index = Math.abs(hash) % tableSize;
```

The number 31 is commonly used because it helps spread string values reasonably well in many practical cases.

## 6.4 Collision

A collision happens when two different keys map to the same index.

Example:

```
tableSize = 10

key 15 -> index 5
key 25 -> index 5
```

Both keys want to be stored at index 5.

### 6.5 Collision Handling

Two common methods are:

| Method | Idea |
|---|---|
| Separate Chaining | Each table index stores a list of entries. |
| Open Addressing | If a position is occupied, search for another position. |

Table 2: Common collision handling methods

For this lab, you will use separate chaining because it is easier to understand and implement.

### 6.6 HashSet vs. HashMap

| Structure | Stores |
|---|---|
| HashSet | Only keys. It is useful for checking whether something exists. |
| HashMap | Key-value pairs. It is useful for counting, mapping, and storing associated data. |

Table 3: HashSet and HashMap comparison

Example:

```
HashSet<String> seen = new HashSet<>();
seen.add("apple");
```

```
HashMap<String, Integer> frequency = new HashMap<>();
frequency.put("apple", 3);
```

## 7 Comparison of Data Structures

| Data Structure | Removal Rule | Best Used When |
|---|---|---|
| Stack | Most recent item first | Matching, undo, expression evaluation. |
| Queue | Earliest item first | Waiting line, request processing. |
| Deque | Both ends | Flexible front/back processing. |
| Priority Queue | Highest or lowest priority first | Ranking, top `k`, scheduling. |

| | | |
|---|---|---|
| Hash Table | Direct access by key | Fast lookup, counting, mapping. |

# 8 Part A: Guided Practice Questions

These tasks are designed to strengthen data structure selection and explanation skills before implementation.

### Task A1: Identify the Best Data Structure

For each problem, identify whether it is best solved using Stack, Queue, Deque, Priority Queue, HashSet, HashMap, Hash Table, or another technique. Briefly justify your answer.

1. Check whether a string of parentheses is valid.
2. Simulate a line of customers waiting for service.
3. Find the top 3 most frequent numbers in an array.
4. Count how many times each word appears in a paragraph.
5. Store student IDs and quickly check whether an ID already exists.
6. Process tasks where each task has a different priority.
7. Implement undo and redo operations in a text editor.
8. Store key-value pairs using your own hash table.

### Task A2: What Is Stored?

Choose 4 problems from Task A1. For each problem, explain:

- what data structure is used,
- what information is stored,
- when information is inserted,
- when information is removed,
- why this structure is suitable.

# 9 Part B: Required Exercises

Complete all required exercises below.

For each exercise, include:

- problem restatement,
- selected data structure,

- explanation of what is stored,
- algorithm idea,
- complexity analysis,
- at least 3 test cases.

## 9.1 Exercise B1: Valid Parentheses

**Question.** Given a string containing only the characters:

```
( ) [ ] { }
```

determine whether the string is valid.

A string is valid if:

- every opening bracket has a matching closing bracket,
- brackets are closed in the correct order,
- no closing bracket appears before its matching opening bracket.

**Example 1.**

```
Input:  "{[()]}"
Output: true
```

**Example 2.**

```
Input:  "{[(])}"
Output: false
```

**Focus.** Stack.

**Students must explain:**

- What is stored in the stack?
- When do we push?
- When do we pop?
- What makes the string invalid?
- What is the time complexity?

## 9.2 Exercise B2: Implement a Circular Queue

**Question.** Implement a circular queue using an array.

Your class should support:

```
enqueue(int value)
dequeue()
front()
isEmpty()
isFull()
```

**Focus.** Queue implementation.

**Required behavior.**

```
capacity = 3

enqueue(10)
enqueue(20)
enqueue(30)
isFull()  -> true
dequeue() -> 10
enqueue(40)
front()   -> 20
```

**Students must explain:**

- What do `front` and `rear` represent?
- Why do we use modulo?
- What is overflow?
- What is underflow?
- Why is Circular Queue better than a simple linear queue?

### 9.3 Exercise B3: Top K Frequent Elements

**Question.** Given an integer array and an integer `k`, return the `k` most frequent elements.

**Example.**

```
Input:  nums = [1, 1, 1, 2, 2, 3], k = 2
Output: [1, 2]
```

**Focus.** HashMap + PriorityQueue.

**Suggested approach.**

1. Use `HashMap<Integer, Integer>` to count frequencies.
2. Use `PriorityQueue` to keep track of frequent elements.
3. Return the top `k` elements.

**Students must explain:**

- What does the HashMap store?
- What does the PriorityQueue store?

- What determines priority?
- Why do we need both structures?
- What is the time complexity?

### 9.4 Exercise B4: Implement a Simple Hash Table

**Question.** Implement a simple hash table using separate chaining.

The hash table should store:

```
String key -> int value
```

Your class should support:

```
put(String key, int value)
get(String key)
remove(String key)
containsKey(String key)
```

**Focus.** Hashing algorithm and collision handling.

**Suggested simple hash function.**

```
private int hash(String key) {
    int hash = 0;

    for (int i = 0; i < key.length(); i++) {
        hash = hash * 31 + key.charAt(i);
    }

    return Math.abs(hash) % table.length;
}
```

**Example.**

```
put("Alice", 90)
put("Bob", 85)
put("Alice", 95)

get("Alice")        -> 95
containsKey("Bob") -> true
remove("Bob")
containsKey("Bob") -> false
```

**Students must explain:**

- How is the hash value computed?
- How is the table index computed?
- What is a collision?
- How does separate chaining handle collisions?
- What happens when two keys map to the same index?
- Why is the average lookup time close to $O(1)$?

# 10 Part C: Further Practice Questions

Choose **2 out of 5** problems below.

For each selected problem, include:

1. problem restatement,
2. selected data structure,
3. explanation of what is stored,
4. algorithm idea,
5. time and space complexity,
6. at least 3 test cases.

## 10.1 C1: Remove Adjacent Duplicates in a String

**Question.** Given a string `s`, repeatedly remove adjacent duplicate characters until no such pair remains.

**Example.**

```
Input:  s = "abbaca"
Output: "ca"
```

Explanation:

```
abbaca -> aaca -> ca
```

**Focus.** Stack.

**Hint.** Use a stack to store characters that have not been removed yet. When the current character is the same as the top of the stack, remove the top. Otherwise, push the current character.

**Students must explain:**

- What does the stack store?
- When do we push?
- When do we pop?
- Why does the stack help avoid repeated scanning?

## 10.2 C2: First Non-Repeating Character in a Stream

**Question.** Given a stream of characters, after each new character arrives, output the first character that has appeared exactly once so far.

If there is no non-repeating character, output `#`.

**Example.**

```
Input:  a a b c
Output: a # b b
```

Step-by-step:

```
After 'a' -> a
After 'a' -> #
After 'b' -> b
After 'c' -> b
```

**Focus.** Queue + HashMap.

**Hint.**

Use:

```
Queue<Character> queue;
HashMap<Character, Integer> frequency;
```

The queue keeps the order of characters. The map keeps the frequency of each character.

**Students must explain:**

- What does the queue store?
- What does the HashMap store?
- Why do we remove characters from the front of the queue?
- Why do we need both structures?

### 10.3 C3: Kth Largest Element in an Array

**Question.** Given an integer array `nums` and an integer `k`, return the kth largest element.

**Example.**

```
Input:  nums = [3, 2, 1, 5, 6, 4], k = 2
Output: 5
```

**Focus.** Priority Queue.

**Hint.** Use a min-priority queue of size `k`. When the queue size becomes greater than `k`, remove the smallest element. At the end, the top of the priority queue is the kth largest element.

**Students must explain:**

- Why do we use a min-priority queue?
- Why is the queue size limited to `k`?
- What does the top of the queue represent?
- Why is this better than sorting the whole array?

### 10.4 C4: Group Anagrams

**Question.** Given an array of strings, group words that are anagrams of each other.

Two words are anagrams if they contain the same characters with the same frequencies.

**Example.**

```
Input: ["eat", "tea", "tan", "ate", "nat", "bat"]

Output:
[
  ["eat", "tea", "ate"],
  ["tan", "nat"],
  ["bat"]
]
```

**Focus.** HashMap + hashing idea.

**Hint.** Use a normalized form of each word as the key. For example, sort the characters:

```
eat -> aet
tea -> aet
ate -> aet
```

Then store words with the same key in the same list.

**Students must explain:**

- What is used as the HashMap key?
- What is used as the HashMap value?
- Why do anagrams have the same key?
- What is the complexity of sorting each word?

### 10.5 C5: Design a Simple Browser History

**Question.** Design a simple browser history system that supports:

```
1 visit(String url)
2 back()
3 forward()
4 current()
```

**Example.**

```
visit("google.com")
visit("youtube.com")
visit("github.com")

back()    -> youtube.com
back()    -> google.com
forward() -> youtube.com
current() -> youtube.com
```

**Focus.** Stack.

**Hint.** Use two stacks:

```
Stack<String> backStack;
Stack<String> forwardStack;
```

When visiting a new page, clear the forward stack.

**Students must explain:**

- What does the back stack store?
- What does the forward stack store?
- Why should the forward stack be cleared after visiting a new page?
- How does this simulate real browser behavior?

# 11 Part D: Reflection and Explanation

### Task D1: Concept Reflection

Write a short reflection answering:

1. What is the difference between Stack and Queue?
2. What is the difference between Queue and Priority Queue?
3. What is the difference between HashSet and HashMap?
4. What is a hash collision?
5. Why does a good hash function matter?
6. Why is separate chaining useful?
7. In Exercise B3, why do we need both HashMap and PriorityQueue?
8. Which required exercise was the hardest, and why?

### Task D2: Explain Without Code

Choose one required exercise from Part B and explain the solution in plain English without showing code.

Your explanation must include:

- the selected data structure,
- what information is stored,
- when data is inserted or removed,
- why the structure is suitable,
- why the complexity is efficient.

## 12 Testing Requirement

For every required exercise:

- include at least 3 test cases,
- include at least 1 edge case,
- show expected output and actual output,
- briefly explain why each test case is useful.

Possible edge cases:

| **Exercise** | **Possible Edge Case** |
|---|---|
| Valid Parentheses | Empty string, only closing bracket, mismatched bracket. |
| Circular Queue | Dequeue from empty queue, enqueue into full queue. |
| Top K Frequent Elements | All elements have same frequency, `k = 1`. |
| Hash Table | Collision case, removing a missing key, updating existing key. |

Table 5: Suggested edge cases

## 13 Deliverables and Submission

Submit one `.zip` file containing:

| **Item** | **Description** |
|---|---|
| Source code | Java files for all required exercises and selected further practice questions. |
| Report PDF | Explanation, selected data structure, stored information, algorithm idea, complexity analysis, and test evidence. |
| Tests | Input/output screenshots or text files. |

## 14 Suggested Grading Breakdown

## 15 Final Notes

Stack, Queue, Priority Queue, and Hash Table are not just library classes. They represent different ways of organizing information.

A strong solution should answer three questions:

1. What should be stored?
2. How should it be retrieved?

| **Criterion** | **Weight** |
|---|---|
| Data structure selection and explanation | 20% |
| Correctness and edge-case handling | 25% |
| Hash table implementation and collision handling | 20% |
| Complexity analysis | 15% |
| Code quality and organization | 15% |
| Report clarity | 5% |

Table 7: Suggested grading breakdown

3. Why does this structure make the solution efficient?

The most important goal is not only to write correct Java code, but also to explain why the chosen data structure is appropriate.